Ateliers GitLab CI/CD

Cahier d'exercices

Introduction....3
Pré-requis....3
Projet Fil rouge....3
Fichiers sources pour les ateliers....3
Utilisation des solutions....3
Atelier 1 : Démarrage de la plateforme....4
Option 1 : Installation locale via Docker....4
Visualisation interface administrateur....4
Option 2 : Utilisation plateforme en ligne....4
Atelier 2 : Workflow de collaboration....5
2.1 Groupes / Projets / Membres....5
2.2 Milestones, Issues, Labels, Tableaux de bord....5
Mise en place des labels, Milestone, et tableaux de bord....5
Création d'issues....5
2.3 MergeRequest....5
1. Création de merge request sur gitlab....6
2. Mise en place environnement de développement + développement....6
3. Pousser les modifications....6
4. Revue de code....6
5. Compléments de développement et mise à jour du dépôt....6
6. Accepter la MR....6
7. Nettoyage local....7
Ateliers 3 : Bases pipelines GitLab....8
3.1 Enregistrement de Runners....8
Runner partagé....8
Runner dédié au projet....9
3.2 Première pipeline et gabarits....9
Ateliers 4 : Syntaxe *.gitlab-ci.yml*....10
4.1 Stages et Job....10
4.2 Artefact et dependencies....10
Pré-requis Jetons SonarQube et NVD....10
Réutilisation d'artefact....11
4.3 Conditions et GITLAB_STRATEGY....11
4.4 Gabarits et inclusions....11
4.5 Construction d'une image Docker et publication dans un registre....12
Pré-requis Activer le registre interne de Gitlab....12
Implémentation de la pipeline....12
4.6 Mise en place d'environnements....12
Ateliers 5 : Support GitLab pour jobs standard....13
5.1 Publication des tests unitaires....13
5.2 Couverture des tests....13
5.3 Analyses statiques....13
5.4 GitLab Registry (Maven)....13
5.5 Release....14
5.6 Kubernetes....14
5.7 AutoDevOps....15

Introduction

Pré-requis

Pour jouer les démos, les éléments suivants sont nécessaires :

- Bonne connexion Internet
- Environnement Linux de préférence
- Docker
- *git* et *gitk*
- Optionnellement cluster Kubernetes : ***kind***

Projet Fil rouge

Le projet fil rouge utilisé dans l'atelier s'appelle ***multi-module***. C'est un projet backend Spring dans le langage Java basé sur l'outil de build Maven
La connaissance du framework, du langage ne sont pas nécessaires.
La compréhension du rôle d'un outil de build comme Maven est un objectif pédagogique de cette formation.

Fichiers sources pour les ateliers

Les fichiers sources sont dans le sous-répertoire TPs du dépôt du support.

2 fichiers sont fournis :

- ***docker-compose.yaml*** permettant de démarrer dans un réseau docker : une instance de gitlab, un gitlab-runner et un instance de sonarqube
- ***multi-module.zip*** : Le projet fil rouge de départ

Utilisation des solutions

Dans un répertoire de travail, exécuter :

```
mkdir multi-module
git clone https://github.com/dthibau/gitlab-solutions.git
```

Pour chaque atelier où il y a un tag, il suffit d'appeler le script *goto.sh* à partir du répertoire gitlab-solutions pour mettre à jour le répertoire multi-module :

```
cd gitlab-solutions
./goto.sh <tag>
```

Le projet multi-module est alors dans l'état du tag correspondant à l'atelier

Atelier 1 : Démarrage de la plateforme

Objectifs : Premier accès, parcours de l'interface utilisateur, mise en place clé SSH.

Option 1 : Installation locale via Docker

Démarrage installation gitlab via Docker :

Visualiser le fichier docker-compose fourni et l'adapter à votre environnement :

```
docker compose up -d
```

Modifier ***/etc/hosts*** ou ***C:\Windows\System32\Drivers\etc\hosts*** afin que ***gitlab.formation.org*** pointe sur localhost :

```
127.0.0.1 gitlab.formation.org
```

Récupérer le mot de passe root avec :

```
sudo docker exec -it gitlab grep 'Password:'
/etc/gitlab/initial_root_password
```

Se logger avec root.

Visualisation interface administrateur

Création de comptes : avec le compte administrateur, mettre en place 2 comptes GitLab :

- ***leader*** : Le prochain mainteneur du projet
- ***developer*** : Un développeur qui effectuera des commits sur le code source

Se connecter avec le compte *developer* et mettre en place une clé SSH.

Option 2 : Utilisation plateforme en ligne

Se créer un compte sur *gitlab.com*, installer sa clé SSH.

Atelier 2 : Workflow de collaboration

Objectifs :

- Comprendre les différents acteurs accédant aux projets
- Visualiser le support pour la planification et le suivi de tâches
- Comprendre le workflow de résolution d'issues

2.1 Groupes / Projets / Membres

Avec le compte *leader* :

- Création d'un groupe de projet ***formation*** et affecter le compte *developer* dans le rôles développeur
- Création d'un projet privé nommé ***multi-module***, en initialisant un dépôt (présence d'un fichier README)

Parcourir les menus du projet multi-module, en particulier *Settings*.

2.2 Milestones, Issues, Labels, Tableaux de bord

Mise en place des labels, Milestone, et tableaux de bord.

En tant que mainteneur de projet, créer 2 milestones :

- Sprint1
- Sprint2

Au niveau groupe, définir les labels suivants :

- In progress
- Review

Définir ensuite un tableau de bord ajoutant des colonnes pour les 2 labels précédents.
Au niveau projet, utiliser les labels par défaut de GitLab, plus :

- API
- DevOps

Création d'issues

Avec le compte reporter :

- Saisir plusieurs issues dont une s'appelant : « CRUD pour delivery-service »
- Tagger avec API
- Saisir quelques commentaires
- Saisir quelques issues techniques :
  - « Mise en place pipeline CI »
  - « Configuration repository »

Les tagger avec DevOps.
Avec le compte owner/mainteneur :

- Mise au planning et affectation
- Tagger les issues

Avec le compte developer, accès au tableau de bord et déplacement du post-it « CRUD pour delivery-service » : To Do → In progress.

2.3 MergeRequest

1. Création de merge request sur gitlab

En tant que développeur sur GitLab, à partir de l'issue 'CRUD pour delivery-service', créer une Merge Request.
La merge request est préfixée par ***Draft*** et a pour effet de créer une branche portant le nom de l'issue.

2. Mise en place environnement de développement + développement

En tant que développeur sur votre poste de travail :

- Installer une clé SSH
- Récupérer la branche de la merge request :

```
git clone <url-ssh-depot>
git checkout <nom-de-branche>
```

- Reprendre le tag 2.3 des solutions (./goto.sh 2.3 dans le répertoire des solutions)

Construire l'application :

```
./mvnw clean package
```

Exécuter l'application :

```
java -jar application/target/multi-module-application-0.0.1-SNAPSHOT.jar
```

Accéder à l'application :

```
http://localhost:8080
```

3. Pousser les modifications

Le développeur pousse les modifications :

```
git add .
git commit -m 'Implémentation CRUD'
git push
```

En tant que *developer* sur GitLab, supprimer le préfixe Draft.

4. Revue de code

En tant que owner/mainteneur, faire une revue de code et ajouter comme commentaire : « Et les tests ? »

5. Compléments de développement et mise à jour du dépôt

Reprise du tag 2.4. (Exécuter les tests et s'assurer qu'ils passent :

```
./mvnw test
```

Pousser les modifications vers GitLab.

6. Accepter la MR

En tant que Owner/Mainteneur, faire une revue de code. Accepter le Merge Request, supprimer la branche et éventuellement un « squash commit ».

7. Nettoyage local

En local, en tant que développeur, supprimer la branche locale et exécuter :

```
git fetch origin
git checkout origin/main
git remote prune origin
git branch -D <branche-mr-locale>
```

Ateliers 3 : Bases pipelines GitLab

Objectifs :
- Comprendre le rôle des runners et voir les différents types de runners
- Prendre en main l'interface pipeline

3.1 Enregistrement de Runners

Nous enregistrons 2 runners :
- Un runner partagé avec un exécuteur docker
- Un runner dédié au projet avec un exécuteur shell

Runner partagé

Avec le login root, aller dans
Admin Area → CI/CD → runners → New Instance Runner.

Indiquer le tag docker.
Cocher également la case Run untagged jobs.

Copier coller la commande d'enregistrement.

Se logger sur le container gitlab-runner :

```
docker exec -it gitlab-runner bash
```

Coller la commande d'enregistrement.

Choisir les choix par défaut et pour la question « exécuteur » choisir ***docker***.
Indiquer également une image par défaut ruby:3.3

Vérifier :
- que le runner s'est bien enregistré dans l'interface administrateur.
- que le runner partagé est disponible pour le projet.
- si une pipeline AutoDevOps a démarré.

Pour que la pipeline s'exécute correctement, éditer le fichier gitlab-runner/config/config.toml

```
sudo gedit $DEPOT/Tps/gitlab-runner/config/config;toml
```

Ajouter ces lignes dans la configuration du runner.docker partagé :

```
privileged = true
links = ["gitlab.formation.org"]
network_mode = "<nom-du-network-docker-compose>"
volumes = ["/var/run/docker.sock:/var/run/docker.sock","/cache"]
```

Pour voir le <nom-du-network-docker-compose>, vous pouvez exécuter :

```
docker network ls
```

Modifier également au niveau global, le degré de concurrence :

```
concurrent = 4
```

Redémarrer le runner et vérifier son redémarrage:

```
docker restart gitlab-runner
docker logs -f gitlab-runner
```

Si tout est OK, relancer la pipeline AutoDevOps

Runner dédié au projet

Avec le compte *leader*, aller dans
Project → Settings → CI/CD → Runner → New Instance Runner.
Indiquer le tag ***shell***.

Se logger sur le container gitlab-runner :

```
docker exec -it gitlab-runner bash
```

Coller la commande d'enregistrement.
Choisir les choix par défaut et pour la question « exécuteur » choisir shell.
Vérifier le bon enregistrement dans la page runners du projet.

3.2 Première pipeline et gabarits

Dans une branche de feature nommée ***CI*** avec le compte *developer*, créer un fichier ***.gitlab-ci.yml*** à partir des templates proposés par GitLab.

Vous pouvez utiliser le projet Java précédent ou le projet dans la technologie de votre choix.
Repository → <feature-branch> → + → .gitlab-ci.yml → Sélectionner le template

Editer le fichier .gitlab-ci.yml pour utiliser une image compatible avec la version de Java :

```
image: maven:3.9-eclipse-temurin-21
```

Et modifier le nom des stages en conséquences.

Committer. Vérifier l'exécution de la pipeline, quel runner a été utilisé ?

Ateliers 4 : Syntaxe *.gitlab-ci.yml*

Objectifs :

- Comprendre les principaux élements de syntaxe de .gitlab-ci.yml
- Implémenter une pipeline CI typique

4.1 Stages et Job

Mettre en place un fichier *.gitlab-ci.yml* qui effectue une phase de compilation et de tests unitaires.
La pipeline peut utiliser l'image : ***eclipse-temurin:21-jdk***

La commande pour compiler est :

```
./mvnw $MAVEN_OPTS compile
```

Pour tester :

```
./mvnw $MAVEN_OPTS test
```

Les options Maven suivantes peuvent être positionnées dans une variable MAVEN_OPTS :

```
-Dhttps.protocols=TLSv1.2 -Dmaven.repo.local=$CI_PROJECT_DIR/.m2/repository
-
Dorg.slf4j.simpleLogger.log.org.apache.maven.cli.transfer.Slf4jMavenTransferListener=W
ARN
-Dorg.slf4j.simpleLogger.showDateTime=true
-Djava.awt.headless=true
```

Observer l'exécution de la pipeline : les dépendances sont-elles cachées ?

4.2 Artefact et dependencies

Pré-requis Jetons SonarQube et NVD

- Se connecter à localhost:9000 avec admin/admin
- Changer le mot de passe
- Dans le Menu en haut à droite : ***My Account → Security***
- Générer un jeton « Global Analysis »
- Déclarer une variable groupe masquée via l'interface GitLab ***SONAR_TOKEN*** qui contient le jeton

Aouter dans le pom.xml la configuration du plugin OWASP :

```
<plugin>
	<groupId>org.owasp</groupId>
	<artifactId>dependency-check-maven</artifactId>
	<version>12.1.0</version>
	<configuration>
		<nvdApiKey>${env.NVD_API_KEY}</nvdApiKey>
		<nvdApiDelay>6000</nvdApiDelay>
		<ossindexAnalyzerEnabled>false</ossindexAnalyzerEnabled>
	</configuration>
</plugin>
```

Aller sur nvd.nist.gov/developers/request-an-api-key pour demander une clé d'API,
La positionner comme variable de projet ***NVD_API_KEY***

Réutilisation d'artefact

Modifier la pipeline afin qu'elle intègre 3 phases :

- 1 phase de **packaging** stockant l'artefact généré :

```
./mvnw clean package
```

- Une phase **d'analyse** incluant 2 jobs :
  - Une analyse de vulnérabilités des dépendances avec la commande Maven :

```
./mvnw -DnvdApiKey=$NVD_API_KEY dependency-check:aggregate
```

  - Une analyse qualité Sonar (nécessite une installation préalable de Sonar) :

```
./mvnw -Dsonar.host.url=http://gitlab-sonar:9000 -Dsonar.token=$
{SONAR_TOKEN} verify sonar:sonar
```

- Une phase de **déploiement** récupérant l'artefact généré et le copiant dans une arborescence variabilisée. Utiliser l'exécuteur shell pour ce job.

4.3 Conditions et GITLAB_STRATEGY

Modifier la phase de déploiement afin de :

- S'assurer que le dépôt n'est pas cloné en positionnant la variable `GIT_STRATEGY`.
- S'assurer qu'il ne s'exécute que si l'on est sur la branche par défaut.

Interdire l'exécution simultanée de 2 pipelines sur des commits différents en annulant le pipeline en cours si un nouveau commit est poussé.

- Positionner la propriété `auto_cancel` au niveau du `workflow`.
- Marquer les jobs pouvant être interrompus avec la propriété `interruptible`.

4.4 Gabarits et inclusions

Créer un nouveau projet nommé ***gabarit***.
Ajouter un fichier ***maven-template.yml*** qui reprend toutes les phases définies dans la pipeline précédente.
Committer dans la branche main.

Dans le projet multi-module, créer une nouvelle branche ***template***, modifier le fichier ***.gitlab-ci.yml*** afin qu'il utilise le gabarit précédent.

Visualiser les gabarits proposés par GitLab.

4.5 Construction d'une image Docker et publication dans un registre

Pré-requis Activer le registre interne de Gitlab

Le registry est déjà exposé sur le port **5050** dans le docker-compose. Il faut juste s'assurer qu'il est activé dans la config GitLab (`./gitlab/config/gitlab.rb`) :

```
registry_external_url 'http://gitlab.formation.org:5050'
gitlab_rails['registry_enabled'] = true
```

Reconfigurer gitlab :

```
docker compose exec gitlab.formation.org gitlab-ctl reconfigure
```

Implémentation de la pipeline

Dans une branche `docker`, récupérer le fichier `Dockerfile` fourni et le placer à la racine du projet. Committer et pousser. Modifier le fichier `.gitlab-ci.yml` en ajoutant un job qui :

- Récupère les artefacts jar et le Dockerfile
- Se connecte sur le **registry interne GitLab**
- Construit une image Docker
- La pousse sur ce registry

Visualiser l'image poussée dans :
GitLab > Deploy > Container Registry

4.6 Mise en place d'environnements

Reprendre la pipeline précédente et définir des jobs de déploiements dans les environnements suivants :

- **Environnement dynamique** reprenant le nom de branche
- **Environnement de QA**, réservé à la branche main, le déploiement est automatique
- **Environnement de production**, réservé à la branche main, le déploiement est manuel

Le déploiement pourra être effectué avec une commande ***docker run***.

Ateliers 5 : Support GitLab pour jobs standard

Objectifs :

- Implémenter les phases classiques d'une pipeline CI/CD

5.1 Publication des tests unitaires

Dans la phase de build, publier le résultat des tests unitaires présents dans :

```
**/target/surefire-reports/*.xml
```

5.2 Couverture des tests

Le rapport de couverture test est fourni par Jacoco dans le répertoire :

```
target/site/jacoco/index.html
```

L'expression régulière permettant d'extraire le résultat est :

```
/Total.*?([0-9]{1,3})%/
```

5.3 Analyses statiques

Inclure les gabarits permettant :

- L'analyse qualité
- La détection de secret
- SAST

L'analyse qualité nécessite un runner partagé avec DockerInDocker :

```
sudo gitlab-runner register -n \
  --url http://gitlab/ \
  --registration-token REGISTRATION_TOKEN \
  --executor docker \
  --description "My Docker Runner" \
  --docker-image "docker:20.10.16" \
  --docker-privileged \
  --docker-volumes "/certs/client"
```

Et éventuellement si démarrage par docker-compose, ajouter dans la configuration :

```
links = ["gitlab.formation.org"]
network_mode = "<nom-du-network-docker-compose>"
```

5.4 GitLab Registry (Maven)

Ajouter un fichier settings.xml comme suit :

```
<settings>
  <servers>
    <server>
      <id>gitlab-maven</id>
      <configuration>
        <httpHeaders>
          <property>
```

```
            <name>Job-Token</name>
            <value>${env.CI_JOB_TOKEN}</value>
          </property>
        </httpHeaders>
      </configuration>
    </server>
  </servers>
</settings>
```

Ajouter dans le fichier pom.xml la configuration de déploiement :

```
<repositories>
  <repository>
    <id>gitlab-maven</id>
    <url>${CI_API_V4_URL}/projects/${CI_PROJECT_ID}/packages/maven</url>
  </repository>
</repositories>
<distributionManagement>
  <repository>
    <id>gitlab-maven</id>
    <url>${CI_API_V4_URL}/projects/${CI_PROJECT_ID}/packages/maven</url>
  </repository>
  <snapshotRepository>
    <id>gitlab-maven</id>
    <url>${CI_API_V4_URL}/projects/${CI_PROJECT_ID}/packages/maven</url>
  </snapshotRepository>
</distributionManagement>
```

Ajouter un script dans le premier job de build :

```
./mvnw -s settings.xml deploy
```

5.5 Release

En reprenant l'exemple des slides, ajouter un job de ***release*** ne s'exécutant que sur la branche par défaut. Faire en sorte que le numéro de version corresponde à celui indiqué dans le fichier pom.xml.
Pour obtenir le numéro de version, utiliser la commande suivante :

```
./mvnw org.apache.maven.plugins:maven-help-plugin:3.1.0:evaluate \
  -Dexpression=project.version -q -DforceStdout
```

5.6 Kubernetes

Installer ***kind***. Modifier la pipeline afin de déployer sur le cluster kind, essayer d'obtenir les bons liens sur les environnements de développement.
Implémenter le ***on_stop*** permettant de nettoyer les ressources lors de la suppression d'une branche.

5.7 AutoDevOps

Renommer le fichier .gitlab-ci.yml et pousser le projet sur gitlab.com.